AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Patryk Wiktor

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

1.1. Zakres i wymagania funkcjonalne 3

1.2. Przewidywane wyniki 4

2. Analiza problemu **5**

2.1. Zastosowanie listy dwukierunkowej 5

2.2. Sposób działania algorytmu i architektura programu 5

2.3. Przykład działania algorytmu krok po kroku 8

2.4. Opis wykorzystywanych narzędzi: Git, GitHub oraz Doxygen 9

3. Projektowanie **11**

3.1. Wykorzystane narzędzia, kompilator oraz biblioteki 11

3.2. Schemat działania algorytmu 11

3.3. Sposób używania narzędzi Doxygen oraz Git/GitHub 13

4. Implementacja **17**

4.1. Implementacja wzorców projektowych i metod listy 17

4.2. Wyniki działania programu C++ 18

4.3. Wyniki testów scenariuszy systemu kontroli wersji Git 19

5. Wnioski **24**

Literatura **26**

Spis rysunków **26**

Spis tabel **27**

Spis listingów **28**

1. Ogólne określenie wymagań

Głównym celem niniejszego projektu jest zaprojektowanie, implementacja oraz przetestowanie dynamicznej struktury danych w postaci listy dwukierunkowej (*ang. doubly linked list*) działającej na stercie w języku C++. Projekt zakłada praktyczne wykorzystanie obiektowych wzorców projektowych: wzorca kreacyjnego Fabryki (*ang. Factory*) oraz wzorca behawioralnego Iteratora (*ang. Iterator*). Dodatkowym celem pracy jest praktyczne zapoznanie się z systemem kontroli wersji Git oraz platformą GitHub, a także opanowanie procesu automatycznego generowania dokumentacji technicznej kodu źródłowego przy użyciu narzędzia Doxygen oraz systemu składu tekstu LaTeX.

1.1. Zakres i wymagania funkcjonalne

W ramach realizowanego zadania programistycznego należy opracować modularną aplikację konsolową w języku C++, w której każda klasa znajduje się w osobnym pliku nagłówkowym (`.h`) oraz źródłowym (`.cpp`). Zaimplementowana lista dwukierunkowa musi realizować następujące funkcjonalności (metody):

- dodanie nowego elementu na początek listy,
- dodanie nowego elementu na koniec listy,
- dodanie nowego elementu pod wskazany indeks,
- usunięcie elementu z początku listy,
- usunięcie elementu z końca listy,
- usunięcie elementu spod wskazanego indeksu,
- wyświetlenie całej zawartości listy od początku do końca,
- wyświetlenie zawartości listy w odwrotnej kolejności (od końca do początku),
- wyświetlenie następnego elementu względem aktualnej pozycji wskaźnika nawigacji,
- wyświetlenie poprzedniego elementu względem aktualnej pozycji wskaźnika nawigacji,
- całkowite wyczyszczenie listy wraz ze zwolnieniem pamięci zaalokowanej na stercie.

Poza samą implementacją struktury danych projekt obejmuje szereg wymagań związanych z inżynierią oprogramowania oraz zarządzaniem kodem źródłowym:

1. **Kontrola wersji (Git oraz GitHub):** utworzenie zdalnego repozytorium na platformie GitHub i regularne rejestrowanie postępów prac (minimum 5 zatwierdzeń – *commitów*). W trakcie prac należy przeprowadzić i udokumentować scenariusze obejmujące: cofnięcie stanu projektu o dwa commity, trwałe usunięcie jednego commita, symulację pracy w dwóch różnych lokalizacjach na dysku (klonowanie, dopisanie metod, synchronizacja poleceniami `push` oraz `pull`), a także celowe usunięcie pliku źródłowego powodujące błąd kompilacji i jego przywrócenie ze zdalnego repozytorium.

2. **Dokumentacja automatyczna (Doxygen):** opatrzenie wszystkich klas, pól oraz metod odpowiednimi komentarzami dokumentacyjnymi i wygenerowanie z nich pełnej dokumentacji technicznej do formatu LaTeX, a następnie skompilowanie jej do pliku PDF.

3. **Dokumentacja projektowa (LaTeX):** sporządzenie niniejszego sprawozdania opisującego strukturę programu, zastosowane algorytmy oraz przebieg pracy z narzędziami Git i Doxygen.

1.2. Przewidywane wyniki

W wyniku realizacji projektu oczekuje się uzyskania w pełni funkcjonalnego, stabilnego programu komputerowego, którego poprawność zostanie zweryfikowana poprzez zestaw testów jednostkowych wywoływanych w funkcji głównej `main()`. Przewiduje się, że zastosowanie alokacji na stercie w połączeniu z odpowiednią obsługą destruktora oraz metody czyszczącej wyeliminuje ryzyko wystąpienia wycieków pamięci (*ang. memory leaks*).

Ponadto efektem końcowym prac będzie uporządkowane repozytorium w serwisie GitHub odzwierciedlające historię rozwoju oprogramowania oraz kompletna, dwuczęściowa dokumentacja w formacie PDF (sprawozdanie projektowe oraz dokumentacja wygenerowana przez program Doxygen).

2. Analiza problemu

2.1. Zastosowanie listy dwukierunkowej

Lista dwukierunkowa jest jedną z fundamentalnych dynamicznych struktur danych stosowanych w informatyce. W przeciwiewieństwie do zwykłej tablicy nie wymaga ona rezerwowania ciągłego obszaru pamięci RAM na starcie programu, a w odróżnieniu od listy jednokierunkowej umożliwia swobodne przemieszczanie się po elementach w obu kierunkach (do przodu oraz wstecz).

Algorytmy oparte na liście dwukierunkowej znajdują szerokie zastosowanie w systemach operacyjnych oraz aplikacjach użytkowych:

- **Przeglądarki internetowe:** obsługa historii przeglądanych stron (przyciski „Wstecz" oraz „Dalej" wymagają szybkiego przejścia do poprzedniego lub następnego węzła).
- **Edytory tekstu i grafiki:** implementacja mechanizmów cofania i ponawiania operacji (*Undo / Redo*), gdzie każda akcja użytkownika stanowi osobny węzeł na liście.
- **Odtwarzacze multimedialne:** tworzenie list odtwarzania (*playlist*), w których użytkownik może przełączać utwory na kolejny lub poprzedni.
- **Systemy operacyjne:** zarządzanie kolejkami procesów, wątków oraz alokacją wolnych bloków pamięci na stercie.
- **Biblioteki standardowe języków programowania:** w języku C++ kontener wzorcowy `std::list` z biblioteki STL jest zaimplementowany właśnie jako lista dwukierunkowa.

2.2. Sposób działania algorytmu i architektura programu

Podstawowym elementem budulcowym listy dwukierunkowej jest węzeł (klasa `Node`). Każdy węzeł tworzony jest dynamicznie na stercie przy użyciu operatora `new` i przechowuje trzy składowe: wartość liczbową (`data`), wskaźnik na kolejny element (`next`) oraz wskaźnik na element poprzedni (`prev`). Strukturę klasy `Node` przedstawia listing 1 (s. 5).

```
1 class Node {
2 public:
3     int data;
```

```
4     Node* next;
5     Node* prev;
6
7     Node(int val);
8 };
```

Listing 1. Deklaracja klasy wezla w pliku Node.h

Główna klasa listy (`DoublyLinkedList`) zarządza strukturą za pomocą trzech wskaźników: `head` (wskazuje pierwszy element listy), `tail` (wskazuje ostatni element listy) oraz `currentNode` (wskaźnik pomocniczy służący do nawigacji krok po kroku). Dodatkowo pole `size` przechowuje aktualną liczbę elementów, co pozwala na walidację indeksów w czasie stałym $O(1)$.

Zgodnie z wymaganiami w projekcie zastosowano dwa wzorce projektowe, a całość podzielono na osobne moduły zestawione w tabeli 2.1 (s. 7):

1. **Wzorzec Fabryki (`NodeFactory`):** hermetyzuje proces tworzenia obiektów na stercie. Zamiast wywoływać operator `new Node(val)` bezpośrednio w metodach listy, klasa `DoublyLinkedList` deleguje to zadanie do statycznej metody `NodeFactory::createNode(int val)`.

2. **Wzorzec Iteratora (`Iterator`):** oddziela logikę przechodzenia po strukturze danych od samej implementacji listy. Klasa `Iterator` przechowuje wskaźnik na bieżący węzeł i udostępnia metody `next()`, `prev()`, `isValid()` oraz `getData()`, co pozwala na bezpieczne wyświetlanie listy w obu kierunkach.

Tab. 2.1. Podział projektu na pliki nagłówkowe i źródłowe

Plik nagłówkowy	**Plik źródłowy**	**Przeznaczenie modułu**
`Node.h`	`Node.cpp`	Reprezentacja pojedynczego węzła listy przechowującego dane oraz wskaźniki `next` i `prev`.
`NodeFactory.h`	`NodeFactory.cpp`	Realizacja wzorca projektowego Fabryki – dynamiczna alokacja węzłów na stercie.
`Iterator.h`	`Iterator.cpp`	Realizacja wzorca projektowego Iteratora – sekwencyjne przeglądanie elementów listy.
`DoublyLinkedList.h`	`DoublyLinkedList.cpp`	Główna klasa listy dwukierunkowej zawierająca 11 wymaganych metod operacyjnych.
–	`main.cpp`	Punkt wejścia programu – testy funkcjonalne wszystkich metod klasy.

2.3. Przykład działania algorytmu krok po kroku

Aby zobrazować sposób działania wskaźników w liście dwukierunkowej, poniżej prześledzono ręcznie wykonanie pięciu przykładowych operacji na liście. Oznaczenie `NULL` symbolizuje pusty wskaźnik (`nullptr`).

1. **Stan początkowy (utworzenie pustej listy):**
   - `head = NULL`, `tail = NULL`, `size = 0`.
2. **Krok 1 – Dodanie elementu 20 na początek (`pushFront(20)`):**
   Fabryka alokuje na stercie węzeł [20]. Ponieważ lista była pusta, nowy węzeł staje się jednocześnie głową i ogonem listy:
   - Węzeł [20]: `prev = NULL`, `next = NULL`.
   - `head` → [20], `tail` → [20], `size = 1`.
3. **Krok 2 – Dodanie elementu 10 na początek (`pushFront(10)`):**
   Alokowany jest węzeł [10]. Jego wskaźnik `next` zostaje podpięty pod dotychczasową głowę ([20]), natomiast wskaźnik `prev` węzła [20] zostaje skierowany na [10]. Następnie wskaźnik `head` przesuwa się na nowy element:
   - Stan listy: `NULL` ← [10] ↔ [20] → `NULL`.
   - `head` → [10], `tail` → [20], `size = 2`.
4. **Krok 3 – Dodanie elementu 40 na koniec (`pushBack(40)`):**
   Alokowany jest węzeł [40]. Wskaźnik `next` dotychczasowego ogona ([20]) zostaje skierowany na [40], a wskaźnik `prev` nowego węzła na [20]. Ogon `tail` przesuwa się na [40]:
   - Stan listy: `NULL` ← [10] ↔ [20] ↔ [40] → `NULL`.
   - `head` → [10], `tail` → [40], `size = 3`.
5. **Krok 4 – Wstawienie elementu 30 pod indeks 2 (`insertAt(2, 30)`):**
   Indeksowany od zera element o indeksie 2 to obecnie węzeł [40]. Nowy węzeł [30] musi zostać wstawiony pomiędzy węzły [20] (indeks 1) oraz [40] (dotychczasowy indeks 2). Następuje przepięcie czterech wskaźników: `prev` nowego węzła wskazuje na [20], a jego `next` na [40]; następnie `next` węzła [20] oraz `prev` węzła [40] zostają skierowane na nowy węzeł [30]:
   - Stan listy: `NULL` ← [10] ↔ [20] ↔ [30] ↔ [40] → `NULL`.

- `head` → [10], `tail` → [40], `size = 4`.

6. **Krok 5 – Usunięcie elementu spod indeksu 1 (`removeAt(1)`):**
   Usuwanym elementem jest węzeł [20]. Algorytm łączy bezpośrednio jego sąsiadów: wskaźnik `next` węzła [10] zostaje przestawiony na [30], a wskaźnik `prev` węzła [30] zostaje przestawiony na [10]. Odpięty węzeł [20] zostaje zwolniony z pamięci sterty instrukcją `delete`:

   - Stan listy: `NULL` ← [10] ↔ [30] ↔ [40] → `NULL`.
   - `head` → [10], `tail` → [40], `size = 3`.

2.4. Opis wykorzystywanych narzędzi: Git, GitHub oraz Doxygen

W procesie wytwarzania oprogramowania zastosowano profesjonalne narzędzia wspierające pracę programisty:

Git oraz platforma GitHub – Git to rozproszony system kontroli wersji (*ang. Distributed Version Control System*) umożliwiający śledzenie historii zmian w plikach źródłowych projektu. Każda zapisana migawka stanu projektu (tzw. *commit*) posiada unikalny identyfikator kryptograficzny SHA-1, autora, znacznik czasu oraz opis zmian. Git operuje na trzech głównych obszarach lokalnych: katalogu roboczym (*Working Directory*), poczekalni (*Staging Area / Index*) oraz lokalnym repozytorium (*.git*). Za pomocą wskaźnika `HEAD` system określa aktualnie wybraną gałąź oraz ostatni commit.

Platforma **GitHub** stanowi natomiast chmurowy serwer hostingowy dla repozytoriów Git. Pozwala ona na przechowywanie kopii zapasowej kodu na serwerze zdalnym (`origin`) oraz synchronizację pracy pomiędzy różnymi lokalizacjami za pomocą poleceń `git clone` (klonowanie repozytorium), `git push` (wysyłanie zatwierdzonych zmian na serwer) oraz `git pull` (pobieranie i scalanie zmian z serwera). W przypadku awarii lub błędów w kodzie Git umożliwia bezpieczne cofanie się w historii (`git checkout`, `git reset`) oraz odzyskiwanie usuniętych plików (`git restore`).

Doxygen – jest to standardowe narzędzie do automatycznego generowania dokumentacji technicznej na podstawie specjalnie sformatowanych komentarzy umieszczonych bezpośrednio w kodzie źródłowym C++. Parser programu Doxygen analizuje strukturę plików nagłówkowych i źródłowych, wyodrębnia bloki komentarzy rozpoczynające się od znaczników `/**` lub `///<` i interpretuje zawarte w nich dyrektywy specjalne, takie jak:

- `@file` – opis modułu (pliku) źródłowego lub nagłówkowego,
- `@class` – przypisanie komentarza do konkretnej klasy,
- `@brief` – skrócony opis przeznaczenia klasy lub metody,
- `@param` – opis pojedynczego parametru wejściowego funkcji,
- `@return` – opis wartości zwracanej przez metodę.

Na podstawie zebranych danych program Doxygen (konfigurowany za pomocą interfejsu graficznego *Doxywizard*) generuje kompletną strukturę dokumentacji w formacie LaTeX oraz HTML, która po kompilacji tworzy gotowy podręcznik referencyjny API projektu w formacie PDF.

3. Projektowanie

3.1. Wykorzystane narzędzia, kompilator oraz biblioteki

Do realizacji projektu wykorzystano następujący zestaw narzędzi programistycznych oraz dokumentacyjnych:

- **Język programowania:** C++ (standard ISO C++17), umożliwiający niskopoziomowe zarządzanie pamięcią na stercie za pomocą operatorów `new` oraz `delete`, a także obiektowe projektowanie architektury w oparciu o klasy.
- **Zintegrowane środowisko programistyczne (IDE):** Microsoft Visual Studio 2022 Community (architektura 64-bitowa, konfiguracja budowania *Debug x64*).
- **Kompilator:** MSVC (*Microsoft Visual C++ Compiler*, narzędzie `cl.exe`) z włączonym standardowym poziomem ostrzeżeń `/W3` oraz trybem zgodności ze standardem C++ (`/permissive-`).
- **Biblioteki dodatkowe:** projekt został zrealizowany wyłącznie w oparciu o bibliotekę standardową języka C++ (`<iostream>` do obsługi strumieni wejścia/wyjścia w konsoli). Nie korzystano z zewnętrznych bibliotek kontenerowych, ponieważ celem zadania była autorska implementacja struktury listy oraz wzorców projektowych od podstaw. W plikach nagłówkowych zastosowano dyrektywę preprocesora `#pragma once`, zapobiegającą wielokrotnemu dołączaniu tych samych nagłówków podczas kompilacji wieloplikowej.
- **System kontroli wersji:** Git (wbudowany moduł *Git Changes* w środowisku Visual Studio 2022 oraz terminal wiersza poleceń) połączony ze zdalnym repozytorium na platformie **GitHub**.
- **Generator dokumentacji i skład tekstu:** program **Doxygen** (wraz z nakładką graficzną *Doxywizard*) oraz środowisko **Overleaf** (LaTeX) do kompilacji dokumentacji referencyjnej API oraz sprawozdania projektowego do formatu PDF.

3.2. Schemat działania algorytmu

Architektura aplikacji została zaprojektowana zgodnie z zasadą pojedynczej odpowiedzialności (*Single Responsibility Principle*).

Kluczowym aspektem działania algorytmu listy dwukierunkowej jest operacja wstawiania oraz usuwania węzła w środku struktury, wymagająca odpowiedniego przepięcia wskaźników `next` oraz `prev` na stercie:

1. **Schemat wstawiania węzła** `newNode` **pomiędzy węzły** A **i** B**:**

`Stan przed: [Węzeł A] <===> [Węzeł B]`

⇓ *1. Alokacja w NodeFactory:* ***`newNode = createNode(val)`***

⇓ *2. Podpięcie nowego węzła:* ***`newNode->prev = A; newNode->next = B;`***

⇓ *3. Przepięcie sąsiadów:* ***`A->next = newNode; B->prev = newNode;`***

`Stan po: [Węzeł A] <===> [newNode] <===> [Węzeł B]`

2. **Schemat usuwania węzła** `toDelete` **znajdującego się pomiędzy** A **i** B**:**

`Stan przed: [Węzeł A] <===> [toDelete] <===> [Węzeł B]`

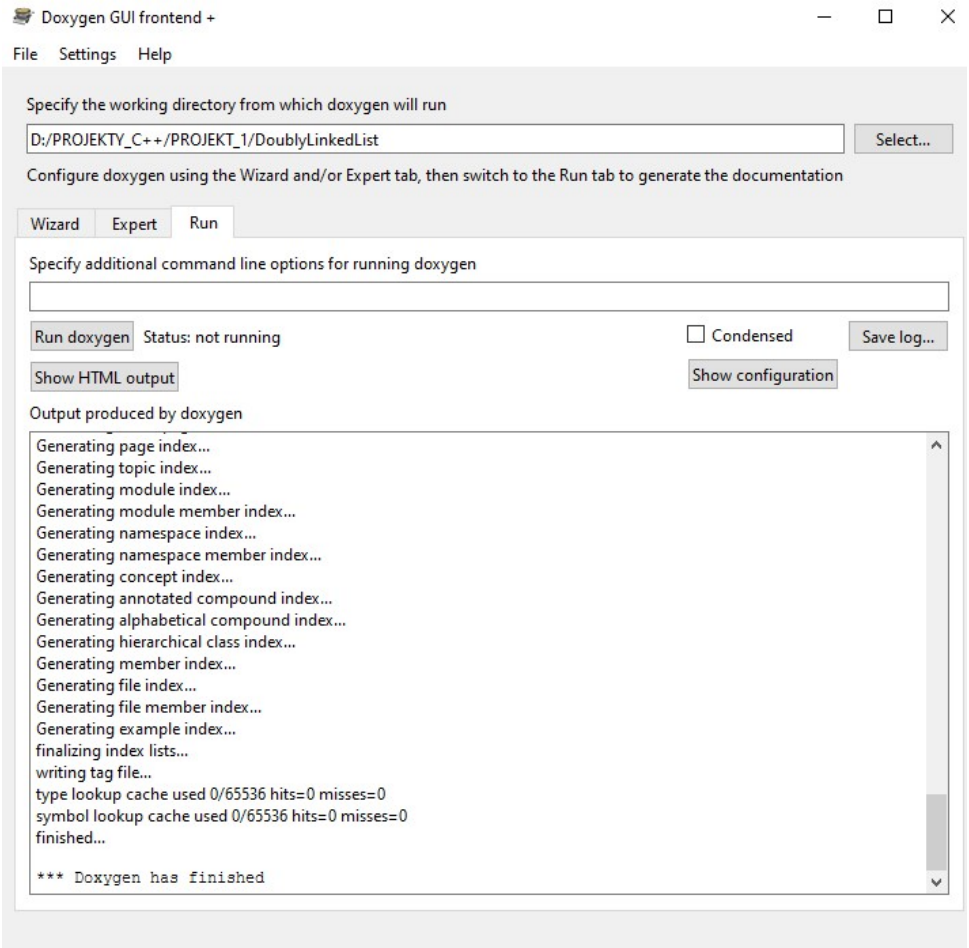
⇓ *1. Pominięcie usuwanego węzła:* ***`A->next = B; B->prev = A;`***

⇓ *2. Zwolnienie pamięci na stercie:* ***`delete toDelete;`***

`Stan po: [Węzeł A] <===> [Węzeł B]`

3.3. Sposób używania narzędzi Doxygen oraz Git/GitHub

Obsługa programu Doxygen: Proces tworzenia automatycznej dokumentacji rozpoczyna się od komentowania kodu w plikach nagłówkowych przy użyciu bloków `/** ... */` oraz znaczników `@file`, `@class`, `@brief`, `@param` i `@return`. Po zakończeniu pisania kodu uruchomiono narzędzie *Doxywizard*, w którym wskazano katalog źródłowy projektu, wybrano tryb ekstrakcji *All Entities* (uwzględniający również pola prywatne klas) oraz zaznaczono format wyjściowy LaTeX. Wygenerowany katalog `latex` został następnie skompilowany z pliku głównego `refman.tex` do wynikowego pliku PDF. Konfigurację oraz przebieg procesu w programie Doxygen przedstawia rysunek 3.1 (s. 13).



Rys. 3.1. Generowanie dokumentacji technicznej projektu w programie Doxywizard

Obsługa systemu Git oraz praca w dwóch lokalizacjach: Przed przystąpieniem do kodowania utworzono zdalne repozytorium na platformie GitHub oraz skonfigurowano plik `.gitignore`, wykluczający ze śledzenia katalogi tymczasowe środowiska Visual Studio (`.vs/`, `x64/`, `Debug/`).

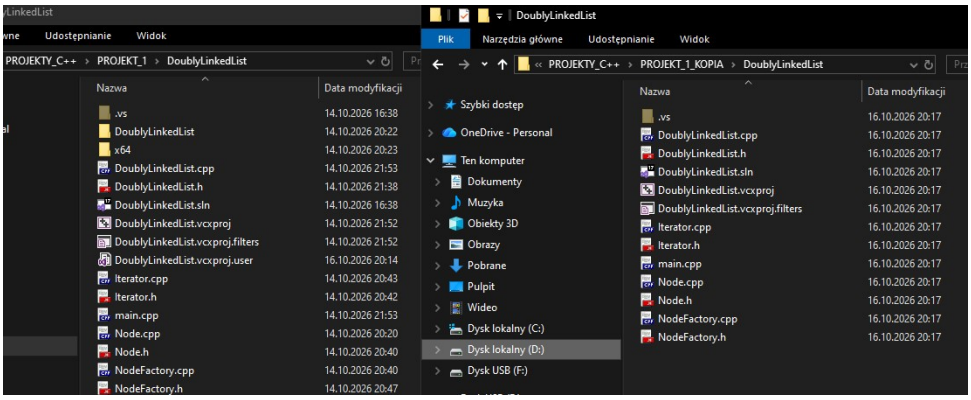
W celu przetestowania rozproszonego charakteru systemu Git przeprowadzono scenariusz symulujący pracę na dwóch niezależnych stanowiskach (w dwóch różnych lokalizacjach na dysku twardym):

1. W pierwszej lokalizacji zaimplementowano klasy bazowe oraz metody dodawania elementów, po czym wysłano je na serwer zdalny (`git push`).

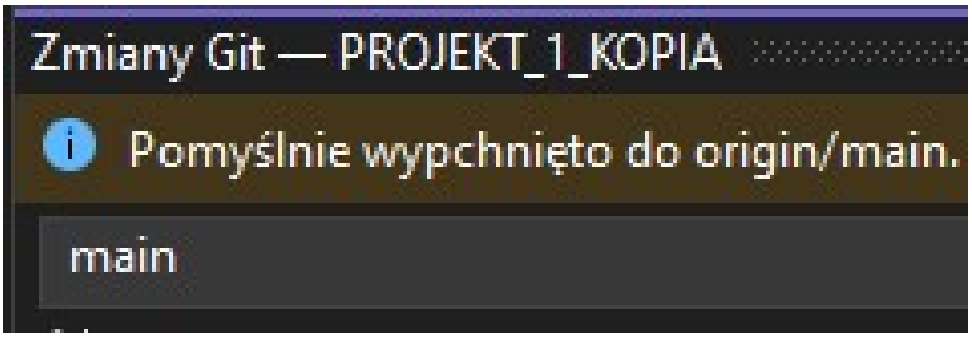
2. Następnie sklonowano repozytorium z serwera GitHub do drugiej lokalizacji na dysku (`git clone`), co obrazuje rysunek 3.2 (s. 14).

3. W pobranym projekcie (druga lokalizacja) dopisano metody usuwania elementów z listy, zatwierdzono zmiany (`git commit`) i wysłano je na serwer zdalny (rysunek 3.3, s. 15).

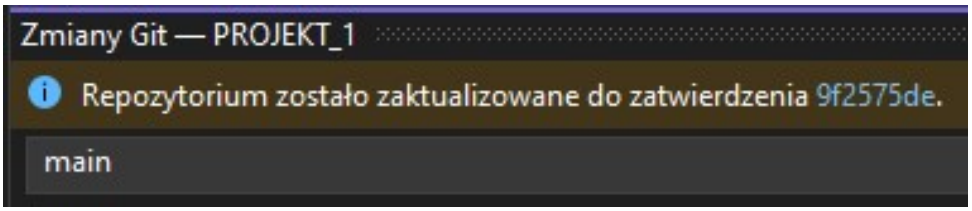
4. W kolejnym kroku powrócono do pierwszej lokalizacji, pobrano aktualne zmiany z serwera poleceniem `git pull` (rysunek 3.4, s. 16), a następnie dopisano metody wyświetlania listy i ponownie zsynchronizowano projekt z serwerem GitHub (rysunek 3.5, s. 16).



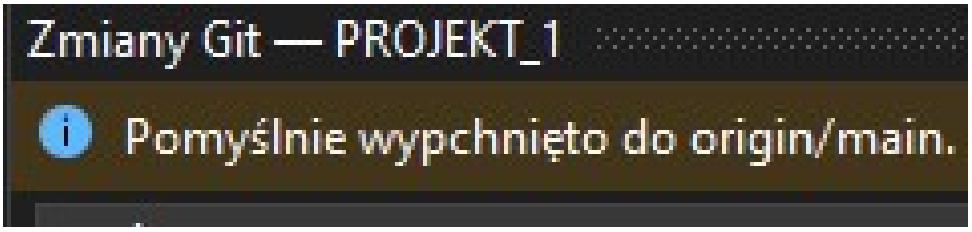
Rys. 3.2. Pobranie (klonowanie) projektu z serwera GitHub do drugiej lokalizacji na dysku



Rys. 3.3. Dopisanie kolejnych metod w drugiej lokalizacji i zapisanie zmian na serwerze



Rys. 3.4. Pobranie zmian z serwera GitHub do pierwszej lokalizacji poleceniem Pull



Rys. 3.5. Dopisanie nowych metod w pierwszej lokalizacji i ponowne wysłanie zmian na serwer

4. Implementacja

4.1. Implementacja wzorców projektowych i metod listy

Zgodnie z założeniami projektowymi tworzenie nowych węzłów na stercie zostało odseparowane od logiki samej listy poprzez zastosowanie wzorca Fabryki (`NodeFactory`). Z kolei bezpieczne przeglądanie struktury zrealizowano za pomocą klasy `Iterator`. Implementację obu wzorców przedstawia listing 2 (s. 17).

```
// NodeFactory.cpp - alokacja wezla na stercie
Node* NodeFactory::createNode(int val) {
    return new Node(val);
}

// Iterator.cpp - przesuwanie wskaznika w przod i w tyl
void Iterator::next() {
    if (current != nullptr) {
        current = current->next;
    }
}

void Iterator::prev() {
    if (current != nullptr) {
        current = current->prev;
    }
}
```

Listing 2. Implementacja wzorca Fabryki oraz Iteratora

Najbardziej złożonymi operacjami w klasie `DoublyLinkedList` są metody manipulujące elementami pod wskazanym indeksem: `insertAt(int index, int val)` oraz `removeAt(int index)`. Obie metody w pierwszej kolejności weryfikują poprawność przekazanego indeksu względem pola `size`. Dla przypadków brzegowych (indeks równy `0` lub koniec listy) delegują one wykonanie do metod `pushFront`/`pushBack` oraz `popFront`/`popBack`. W przypadku operacji wewnątrz listy pętla `for` ustawia wskaźnik pomocniczy na docelowym węźle, po czym następuje przepięcie wskaźników sąsiadów oraz – przy usuwaniu – zwolnienie pamięci operatorem `delete`. Implementację tych metod prezentuje listing 3 (s. 17).

```
void DoublyLinkedList::insertAt(int index, int val) {
    if (index < 0 || index > size) {
        cout << "Blad: Nieprawidlowy indeks" << endl;
        return;
    }
```

```
6       if (index == 0) { pushFront(val); return; }
7       if (index == size) { pushBack(val); return; }
8
9       Node* newNode = NodeFactory::createNode(val);
10      Node* temp = head;
11      for (int i = 0; i < index; i++) {
12          temp = temp->next;
13      }
14
15      newNode->prev = temp->prev;
16      newNode->next = temp;
17      temp->prev->next = newNode;
18      temp->prev = newNode;
19      size++;
20  }
21
22  void DoublyLinkedList::removeAt(int index) {
23      if (index < 0 || index >= size) {
24          cout << "Blad: Nieprawidlowy indeks" << endl;
25          return;
26      }
27      if (index == 0) { popFront(); return; }
28      if (index == size - 1) { popBack(); return; }
29
30      Node* toDelete = head;
31      for (int i = 0; i < index; i++) {
32          toDelete = toDelete->next;
33      }
34      if (currentNode == toDelete) {
35          currentNode = toDelete->next;
36      }
37
38      toDelete->prev->next = toDelete->next;
39      toDelete->next->prev = toDelete->prev;
40      delete toDelete;
41      size--;
42  }
```

Listing 3. Implementacja metod insertAt oraz removeAt w pliku DoublyLinkedList.cpp

4.2. Wyniki działania programu C++

Wszystkie 11 metod klasy `DoublyLinkedList` przetestowano w funkcji `main()`. Przeprowadzony test obejmował kolejno: wstawianie elementów na początek, koniec

oraz pod indeks 2, wyświetlenie listy w obu kierunkach za pomocą iteratora, nawigację po sąsiednich węzłach (`displayNext`, `displayPrev`), usuwanie ze środka, początku i końca, a także całkowite wyczyszczenie listy metodą `clear()`. Wynik wygenerowany przez program w konsoli przedstawia listing 4 (s. 19).

```
1. Test dodawania elementow
Lista (od poczatku): 10 20 30 40
Lista (od konca): 40 30 20 10

2. Test nawigacji (nastepny / poprzedni)
Nastepny element: 30
Nastepny element: 40
Poprzedni element: 30

3. Test usuwania elementow
Usuwanie spod indeksu 1 (wartosc 20):
Lista (od poczatku): 10 30 40
Usuwanie z poczatku (wartosc 10):
Lista (od poczatku): 30 40
Usuwanie z konca (wartosc 40):
Lista (od poczatku): 30

4. Test czyszczenia calej listy
Lista jest pusta.
```

Listing 4. Wynik dzialania programu testowego w konsoli

4.3. Wyniki testów scenariuszy systemu kontroli wersji Git

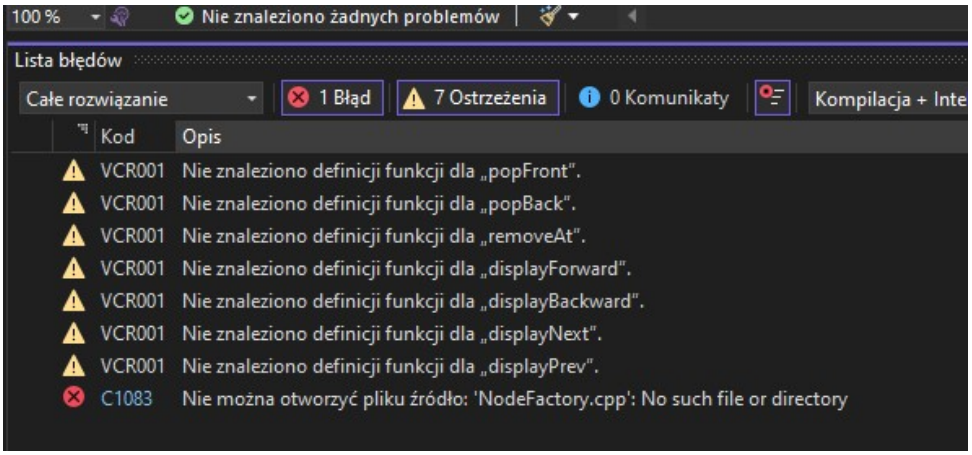
W ramach realizacji projektu przetestowano i udokumentowano wymagane scenariusze awaryjne oraz manipulację historią zmian w systemie Git:

1. **Usunięcie pliku projektu i przywrócenie go z repozytorium:**
   Z katalogu roboczego usunięto plik źródłowy `NodeFactory.cpp`, co doprowadziło do błędu kompilacji i konsolidacji programu w środowisku Visual Studio (rysunek 4.1, s. 20). Następnie za pomocą polecenia `git checkout HEAD -- DoublyLinkedList/NodeFactory.cpp` (lub `git restore`) przywrócono brakujący plik z ostatniego zatwierdzonego stanu repozytorium, dzięki czemu projekt ponownie kompilował się poprawnie (rysunek 4.2, s. 20).

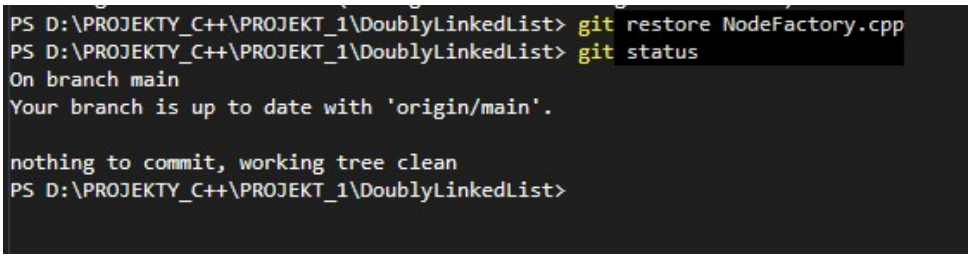
2. **Cofnięcie się o dwa commity:**
   Sprawdzono historię zatwierdzeń komendą `git log --oneline`, a następnie wykonano polecenie `git checkout HEAD~2` (rysunek 4.3, s. 21). Operacja

ta przełączyła repozytorium w stan odłączonego wskaźnika głowy (*detached HEAD*) i cofnęła pliki projektu o dwie wersje wstecz, co zweryfikowano na rysunku 4.4 (s. 21), po czym powrócono na aktualną gałąź główną komendą `git checkout main`.

3. **Trwałe usunięcie jednego commita:**
   Utworzono dodatkowy commit testowy (rysunek 4.5, s. 22), a następnie trwale usunięto go z historii gałęzi za pomocą twardego resetu `git reset --hard HEAD~1` (rysunek 4.6, s. 22). Zaktualizowany stan historii przesłano na serwer GitHub z flagą wymuszenia `git push --force` (rysunek 4.7, s. 23).



Rys. 4.1. Błąd kompilacji projektu po celowym usunięciu pliku źródłowego z katalogu roboczego



Rys. 4.2. Przywrócenie usuniętego pliku z repozytorium Git i przywrócenie pełnej kompilowalności projektu

```
PS D:\PROJEKTY_C++\PROJEKT_1\DoublyLinkedList> git log --oneline
01b8e7c (HEAD -> main, origin/main, origin/HEAD) Dodano testy wszystkich metod listy w main
82ba134 Dodano metody wyswietlania elementow w pliku DoublyLinkedList (pierwsza lokalizacja)
9f2575d Dodanie metod usuwania elementow w pliku DoublyLinkedList (druga lokalizacja)
65961a4 Plik DoublyLinkedList z metodami wstawiania elementow
d8b07ab Klasa Iterator
70f2406 Klasa Node i wzorzec NodeFactory
0438c37 Initial commit
PS D:\PROJEKTY_C++\PROJEKT_1\DoublyLinkedList>
```

Rys. 4.3. Podgląd historii commitów oraz wykonanie cofnięcia o dwa commity wstecz (git checkout HEAD~2)

```
PS D:\PROJEKTY_C++\PROJEKT_1\DoublyLinkedList> git checkout HEAD~2
Note: switching to 'HEAD~2'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 9f2575d Dodanie metod usuwania elementow w pliku DoublyLinkedList (druga lokalizacja)
PS D:\PROJEKTY_C++\PROJEKT_1\DoublyLinkedList> git log --oneline
9f2575d (HEAD) Dodanie metod usuwania elementow w pliku DoublyLinkedList (druga lokalizacja)
65961a4 Plik DoublyLinkedList z metodami wstawiania elementow
d8b07ab Klasa Iterator
70f2406 Klasa Node i wzorzec NodeFactory
0438c37 Initial commit
PS D:\PROJEKTY_C++\PROJEKT_1\DoublyLinkedList>
```

Rys. 4.4. Weryfikacja stanu historii po cofnięciu o dwa commity oraz powrót na gałąź główną (main)

```
PS D:\PROJEKTY_C++\PROJEKT_1\DoublyLinkedList> git log --oneline
5cde1f2 (HEAD -> main, origin/main, origin/HEAD) Testowa funkcjonalnosc
01b8e7c Dodano testy wszystkich metod listy w main
82ba134 Dodano metody wyswietlania elementow w pliku DoublyLinkedList (pierwsza lokalizacja)
9f2575d Dodanie metod usuwania elementow w pliku DoublyLinkedList (druga lokalizacja)
65961a4 Plik DoublyLinkedList z metodami wstawiania elementow
d8b07ab Klasa Iterator
70f2406 Klasa Node i wzorzec NodeFactory
0438c37 Initial commit
```

Rys. 4.5. Utworzenie testowego commita przeznaczonego do usunięcia z historii projektu

```
PS D:\PROJEKTY_C++\PROJEKT_1\DoublyLinkedList> git reset --hard HEAD~1
HEAD is now at 01b8e7c Dodano testy wszystkich metod listy w main
PS D:\PROJEKTY_C++\PROJEKT_1\DoublyLinkedList> git log --oneline
01b8e7c (HEAD -> main) Dodano testy wszystkich metod listy w main
82ba134 Dodano metody wyswietlania elementow w pliku DoublyLinkedList (pierwsza lokalizacja)
9f2575d Dodanie metod usuwania elementow w pliku DoublyLinkedList (druga lokalizacja)
65961a4 Plik DoublyLinkedList z metodami wstawiania elementow
d8b07ab Klasa Iterator
70f2406 Klasa Node i wzorzec NodeFactory
0438c37 Initial commit
PS D:\PROJEKTY_C++\PROJEKT_1\DoublyLinkedList>
```

Rys. 4.6. Trwałe usunięcie ostatniego commita za pomocą polecenia git reset –hard HEAD˜1

```
PS D:\PROJEKTY_C++\PROJEKT_1\DoublyLinkedList> git log --oneline
01b8e7c (HEAD -> main) Dodano testy wszystkich metod listy w main
82ba134 Dodano metody wyswietlania elementow w pliku DoublyLinkedList (pierwsza lokalizacja)
9f2575d Dodanie metod usuwania elementow w pliku DoublyLinkedList (druga lokalizacja)
65961a4 Plik DoublyLinkedList z metodami wstawiania elementow
d8b07ab Klasa Iterator
70f2406 Klasa Node i wzorzec NodeFactory
0438c37 Initial commit
PS D:\PROJEKTY_C++\PROJEKT_1\DoublyLinkedList> git push --force
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/DenisLaBryga/P1_DoublyLinkedList
 + 5cde1f2...01b8e7c main -> main (forced update)
PS D:\PROJEKTY_C++\PROJEKT_1\DoublyLinkedList> git log --oneline
01b8e7c (HEAD -> main, origin/main, origin/HEAD) Dodano testy wszystkich metod listy w main
82ba134 Dodano metody wyswietlania elementow w pliku DoublyLinkedList (pierwsza lokalizacja)
9f2575d Dodanie metod usuwania elementow w pliku DoublyLinkedList (druga lokalizacja)
65961a4 Plik DoublyLinkedList z metodami wstawiania elementow
d8b07ab Klasa Iterator
70f2406 Klasa Node i wzorzec NodeFactory
0438c37 Initial commit
PS D:\PROJEKTY_C++\PROJEKT_1\DoublyLinkedList>
```

Rys. 4.7. Aktualizacja historii na zdalnym serwerze GitHub po usunięciu commita (git push –force)

5. Wnioski

Przeprowadzony projekt pozwolił na praktyczną realizację dynamicznej struktury listy dwukierunkowej działającej na stercie w języku C++ oraz pogłębienie umiejętności posługiwania się narzędziami wspierającymi inżynierię oprogramowania. Na podstawie wykonanych prac sformułowano następujące wnioski końcowe:

1. **Zarządzanie pamięcią i złożoność obliczeniowa listy dwukierunkowej:**
   Dzięki utrzymywaniu dwóch wskaźników skrajnych (`head` oraz `tail`) operacje dodawania i usuwania elementów na początku oraz na końcu listy wykonywane są w czasie stałym $O(1)$, bez konieczności przewijania całej struktury. Operacje odwołujące się do dowolnego indeksu (`insertAt`, `removeAt`) charakteryzują się złożonością liniową $O(n)$. Alokacja węzłów na stercie zapewnia elastyczny rozmiar struktury ograniczony jedynie dostępną pamięcią RAM, jednak wymaga rygorystycznego zwalniania pamięci instrukcją `delete` przy każdym usuwaniu węzła oraz w destruktorze klasy, co skutecznie chroni program przed wyciekami pamięci.

2. **Zastosowanie wzorców projektowych i modularności kodu:**
   Podział programu na osobne pliki nagłówkowe (`.h`) i źródłowe (`.cpp`) dla każdej klasy znacząco poprawia czytelność architektury oraz przyspiesza kompilację przy wprowadzaniu zmian. Zastosowanie wzorca kreacyjnego **Fabryki** (`NodeFactory`) pozwoliło odseparować proces tworzenia obiektów na stercie od logiki samej listy, natomiast wzorzec behawioralny **Iteratora** (`Iterator`) umożliwił bezpieczne przeglądanie elementów w obu kierunkach bez bezpośredniego eksponowania wewnętrznych wskaźników węzłów.

3. **Praca z systemem kontroli wersji Git oraz platformą GitHub:**
   Przetestowane scenariusze praktyczne wykazały kluczową rolę systemu kontroli wersji w codziennej pracy programisty. Mechanizmy takie jak `git checkout` czy `git restore` pozwalają na błyskawiczne odzyskanie przypadkowo usuniętych plików źródłowych i przywrócenie kompilowalności projektu. Polecenia manipulujące historią (`git checkout HEAD~2` oraz `git reset --hard`) umożliwiają swobodną inspekcję wcześniejszych wersji programu oraz wycofywanie błędnych zatwierdzeń. Z kolei synchronizacja za pomocą komend `git clone`, `git push` oraz `git pull` gwarantuje spójność kodu podczas pracy w wielu różnych lokalizacjach. Istotnym elementem konfiguracji okazał się rów-

nież plik `.gitignore`, zapobiegający blokowaniu repozytorium przez tymczasowe pliki robocze środowiska Visual Studio.

4. **Automatyzacja dokumentacji za pomocą narzędzia Doxygen i LaTeX:** Stosowanie ustrukturyzowanych komentarzy dokumentacyjnych bezpośrednio w kodzie źródłowym pozwala na błyskawiczne wygenerowanie pełnej dokumentacji technicznej API. Połączenie generatora Doxygen z systemem składu tekstu LaTeX (w środowisku Overleaf) zapewnia wysoką, publikacyjną jakość wynikowych dokumentów PDF przy minimalnym nakładzie pracy ręcznej podczas aktualizacji kodu.

Spis rysunków

3.1. Generowanie dokumentacji technicznej projektu w programie Doxywizard . . . 13
3.2. Pobranie (klonowanie) projektu z serwera GitHub do drugiej lokalizacji na dysku . . . 14
3.3. Dopisanie kolejnych metod w drugiej lokalizacji i zapisanie zmian na serwerze . . . 15
3.4. Pobranie zmian z serwera GitHub do pierwszej lokalizacji poleceniem Pull . . . 16
3.5. Dopisanie nowych metod w pierwszej lokalizacji i ponowne wysłanie zmian na serwer . . . 16
4.1. Błąd kompilacji projektu po celowym usunięciu pliku źródłowego z katalogu roboczego . . . 20
4.2. Przywrócenie usuniętego pliku z repozytorium Git i przywrócenie pełnej kompilowalności projektu . . . 20
4.3. Podgląd historii commitów oraz wykonanie cofnięcia o dwa commity wstecz (git checkout HEAD~2) . . . 21
4.4. Weryfikacja stanu historii po cofnięciu o dwa commity oraz powrót na gałąź główną (main) . . . 21
4.5. Utworzenie testowego commita przeznaczonego do usunięcia z historii projektu . . . 22
4.6. Trwałe usunięcie ostatniego commita za pomocą polecenia git reset –hard HEAD~1 . . . 22
4.7. Aktualizacja historii na zdalnym serwerze GitHub po usunięciu commita (git push –force) . . . 23

Spis tabel

2.1. Podział projektu na pliki nagłówkowe i źródłowe 7

Spis listingów

1. Deklaracja klasy wezla w pliku Node.h 5
2. Implementacja wzorca Fabryki oraz Iteratora 17
3. Implementacja metod insertAt oraz removeAt w pliku DoublyLinkedList.cpp 17
4. Wynik działania programu testowego w konsoli 19